



Red Hat Lightspeed cost management self-managed Beta Documentation

Updated September 18 2026 at 10:41 PM - English ▼

Table of Contents

- Red Hat Lightspeed cost management self-managed
 - What is Red Hat Lightspeed cost management
 - How can I use Red Hat Lightspeed cost management
 - Cost Management managed vs self-managed
- Prerequisites for Red Hat Lightspeed cost management self-managed
 - Prepare your external infrastructure
 - Create Keycloak users
 - Create Keycloak UI client
 - Create Keycloak service account
- Install Red Hat Lightspeed cost management self-managed
 - Add the Cost Management Service Operator to the software catalog
 - Install the Cost Management Service Operator
 - Configure the Cost Management Service Operator
 - Configure the Cost Management UI OAuth Client
 - Create the UI OAuth client Secret for the Cost Management Service Operator
 - Verify the Cost Management Service Operator
- Client clusters setup
 - Install the Cost Management Metrics Operator
 - Create service accounts to authenticate the clients
 - Configure clients to upload data to the server
- Access the self-managed instance

Red Hat Lightspeed cost management self-managed

What is Red Hat Lightspeed cost management?

Red Hat Lightspeed cost management simplifies managing resources and costs across cloud and OpenShift Container Platform environments, helping system administrators optimize spending and align IT costs with business priorities.

You can use the cost management service to perform the following tasks to help your organization optimize costs, increase efficiency, and save money:

- Visualize, understand, and analyze how your resources are used.
- Track cost trends.
- Break down charges to your projects and organizations.
- Use cost models to apply cost to OpenShift usage metrics or add markups, or both.
- Forecast your future consumption.
- Identify patterns of usage that you might want to investigate.
- Export data to integrate with third-party tools.

How can I use Red Hat Lightspeed cost management?

Red Hat has offered a managed version of cost management for years. For more information on the managed version of cost management, see the Red Hat Lightspeed cost management documentation.

Red Hat is now introducing a self-managed version of cost management. This version is a fully local, self-managed deployment of the Red Hat Lightspeed cost management service running directly inside your own cluster. With this deployment of the cost management service, you are fully in control of your own data and your configuration.

NOTE: Red Hat Lightspeed cost management self-managed is included with the Red Hat OpenShift subscription. ROSA and ARO PAYG customers are also entitled to Red Hat Lightspeed cost management self-managed.

Cost Management managed vs self-managed

The managed and self-managed versions of cost management both provide the essential functions and resources available through the cost management service. Red Hat is in the process of configuring both versions to include the entire portfolio of cost management features. Cost management self-managed provides users additional freedoms within their environment.

NOTE: The high-touch beta release of Red Hat Lightspeed cost management self-managed does not support cloud (AWS, Azure, GCP) costs. This functionality will be added soon.

Advantages of cost management self-managed

- Enables highly regulated, disconnected networks (defense, government, finance) to track infrastructure spending without violating strict internet bans.
- Ensures sensitive data, cluster metrics, and internal workload labels never cross the corporate firewall.
- Enables cost management for organizations that require self-managed infrastructure but need the same visibility available in SaaS deployments.
- Allows for a custom data retention period (vs 90 days on the managed service)
- Allows you to provide your own currency exchange API key

NOTE: The process of setting up self-managed cost management requires you to bring in several aspects of your own external infrastructure. These infrastructure elements include PostgreSQL 16, cache, Kafka, S3-compatible object storage, and OIDC.

Prerequisites for Red Hat Lightspeed cost management self-managed

Prepare your external infrastructure

To set up cost management self-managed, you must provide and manage the external services that cost management depends on while the operator deploys and wires the cost management application components.

NOTE: The cost management operator validates and connects to those services, rather than provisioning them on your behalf.

External customer-owned infrastructure includes:

- PostgreSQL 16
- Redis-compatible cache
- Kafka
- S3-compatible object storage
- OIDC

NOTE: The cost management operator still owns the application stack itself, including the cost management services, RBAC services, UI, ingress, and gateway configuration.

Before beginning the process for setting up and configuring cost management self-managed, make sure the following external infrastructure already exists and is reachable from your cluster. Also ensure that you have access to all of the required information listed below.

PostgreSQL 16

You must provide:

- An existing PostgreSQL 16 service
- The host, port, and SSL mode the application should use
- A Secret containing database credentials that can be used with the `CostManagementServiceConfig` which you will create later
- Pre-created application databases, users, and grants for the enabled components

IMPORTANT: Do not create a cost management database named `postgres`. That is PostgreSQL's maintenance database. No cost management service connects to it.

Current application database names:

- Cost-only path (`ros.enabled: false`):
 - `costonprem_koku` (Koku API, Masu, Listener, Celery, koku migrate)
 - `costonprem_rbac` (RBAC API, worker, RBAC migrate)
- ROS-enabled path adds:
 - `costonprem_ros`
 - `costonprem_kruize`

NOTE:

- *The operator does not treat external PostgreSQL as a create-databases workflow*
- *Bundled PostgreSQL can bootstrap the application database names for dev/CI by connecting to `postgres`; that is a bootstrap convenience only*
- *External PostgreSQL should be documented as pre-provisioned*

Application Secret keys for databases:

- `koku-user` / `koku-password`
- `rbac-user` / `rbac-password`
- `ros-user` / `ros-password` and `kruize-user` / `kruize-password`

NOTE: ROS Secret keys are required, even though ROS is not supported in this initial Beta release.

NOTE: The cost management operator connects to an existing PostgreSQL 16 service. It does not create databases on your external PostgreSQL instance as part of the normal production path. Create `costonprem_koku` and `costonprem_rbac` (and the ROS databases only if you enable ROS) and grant the corresponding users access before applying the `CostManagementServiceConfig`. You do not need a database named `postgres` for cost management.

Cache

You must provide:

- An existing Redis- or Valkey-compatible cache service
- The host and port the application should use
- A Secret with cache credentials when authentication is enabled
- A CA Secret when private TLS trust is required

Kafka

You must provide:

- An existing Kafka service
- The bootstrap server addresses
- SASL credentials when SASL is enabled
- A CA Secret when private TLS trust is required
- The required Kafka topics already created

Required Kafka topic:

- Cost-only path: `platform.upload.announce`

NOTE: The cost management operator validates that the required Kafka topics exist, but it does not create them for you as part of the default production path.

S3-Compatible Object Storage

You must provide:

- An existing S3-compatible object storage service
- The endpoint, port, and TLS mode
- A Secret with access credentials
- A CA Secret when private TLS trust is required
- The bucket names the application will use

Current bucket contract:

- `spec.objectStorage.buckets.koku`
- `spec.objectStorage.buckets.ingress`
- `spec.objectStorage.buckets.ros` when `ros.enabled: true`

NOTE: The cost management operator expects the required buckets to already exist and be accessible with the provided credentials.

OIDC

You must provide:

- An existing external OIDC provider
- A reachable OIDC base URL
- An issuer URL when the token issuer differs from the internal URL
- A CA Secret when private TLS trust is required
- The UI OAuth client Secret in the `CostManagementServiceConfig` namespace
- Keycloak human users (browser login) and a service account client (Cost Management Metrics Operator uploads),

NOTE: Your OIDC setup will vary based on your organisation's authorisation needs. Refer to [Configure the Cost Management UI OAuth client](#) for an example setup.

Create Keycloak users

IMPORTANT: Configure Keycloak manually in the Admin Console. If your environment already has pre-created clients or users, remove or ignore them when they conflict with the steps in this section.

Cost management reads the **access token**. Human users must exist in the **kubernetes realm**, unless you set a different `spec.auth.keycloak.realm` on the `CostManagementServiceConfig`. The Admin Console often opens on the **master realm** after login. Anything you create in master is ignored by cost management.

Use bare identifiers for `org_id` and `account_number`, for example, `1234567` and `1234567`. Do not prefix `org_id` with `org`. A value such as `org1234567` becomes `orgorg1234567` and breaks tenant routing.

Procedure

1. From the OpenShift Container Platform web console, navigate to **Networking > Routes**.
2. From the **Project** dropdown menu, select the project namespace for your keycloak instance.

3. Locate and click the corresponding link under **Location** to login to keycloak.
4. Login to keycloak and ensure you select the **kubernetes realm** from the top left drop down.
5. Navigate to **Realm roles**. Create the **org-admin realm** role if it does not already exist.
6. Navigate to **Users**, and add a new user.
7. Fill out the form ensuring to add an email address, and enable the **Email verified** toggle.
8. Click **create**.
9. Configure the following for the keycloak user:
NOTE: You must enable unmanaged attributes in keycloak realm in order to configure.
 - Add two new **Attributes**; **org_id** and **account_number**, pick your own stable identifiers for these following this example: 1234567 / 1234567
 - **Credentials**; set a password for the new user.
 - **Role mapping**; assign the **org-admin** role.
10. Repeat the user creation process (skipping the org-admin assignment) to create any number of cost management ready user accounts.

Create Keycloak UI client

The Cost Management UI uses oauth2-proxy and a **confidential** OpenID Connect client. This client is **not** the Metrics Operator client. Only this client gets a redirect URI.

You need three values that match the `CostManagementServiceConfig` you will create later:

- `CR_NAME` – metadata.name (example: cost-onprem)
- `NAMESPACE` – metadata.namespace (example: cost-onprem)
- `APPS_DOMAIN` – the cluster apps domain, the same value you will set on `spec.global.clusterDomain` (example: apps.cluster.example.com)

Procedure

1. Open the Keycloak Admin Console in the **Kubernetes** realm.
2. Navigate to **Clients** > **Create client**.
3. Configure the following general settings:
 - Client type: **OpenID Connect**
 - Client ID: a stable name such as cost-management-ui
4. Configure **Capability config**:
 - Client authentication: **On**
 - Standard flow: **On**
 - Direct access grants: **Off**
 - Implicit flow: **Off**

- Service account roles: **Off**
- 5. Under **Login settings**, set **Valid redirect URIs** to exactly one URI:
 - `https://{CR_NAME}-ui-{NAMESPACE}.{APPS_DOMAIN}/oauth2/callback`
- 6. Set **Web origin** to the UI origin with **no path**:
 - `https://{CR_NAME}-ui-{NAMESPACE}.{APPS_DOMAIN}`
- 7. Click **Save**.
- 8. Open **Credentials** and copy the client secret. You will use this secret in a later section (`client_id` / `client-secret` with hyphens).
- 9. Navigate to **Client scopes** and open the dedicated scope named `{client-id}-dedicated`.
- 10. Navigate to **Mappers**.
- 11. Add three mappers and, for each one, set **Add to access token** to **On**. Leave **Add to ID token** and **Add to userinfo** set to **Off**. Follow the mapper examples below to configure:
 - **Mapper 1: Audience**
(Add mapper > **By configuration** > **Audience**)
 - Name: `aud-cost-management-operator`
 - **Included Client Audience**: `cost-management-operator`
 - **Included Custom Audience**: `cost-management-operator` (set both; an Audience mapper with empty audience fields does not set `aud`)
 - **Mapper 2: User Attribute `org_id`**
(Add mapper > **By configuration** > **User Attribute**)
 - Name, User Attribute, and Token Claim Name: `org_id`
 - Claim JSON Type: String
 - **Mapper 3: User Attribute `account_number`**
(Add mapper > **By configuration** > **User Attribute**)
 - Same as mapper 2, with `account_number` in every name field
 - Name, User Attribute, and Token Claim Name: `account_number`
 - Claim JSON Type: String

*NOTE: If Valid redirect URIs does not match the UI Route exactly, login fails after the UI loads even when **UIReady** is True*

Create Keycloak service account

Create a new OpenID Connect client for the Cost Management Metrics Operator on your external Keycloak. Configure the new OIDC client in the Admin Console.

NOTE: Repository install scripts do not replace this step for production.

*IMPORTANT: This client is **not** the same as the UI client (the UI client uses Standard flow **On** and Service accounts **Off**).*

Access tokens from this client must include:

- aud intersecting the operator defaults `cost-management-operator` and/or `cost-management-ui` (unless you override `spec.auth.keycloak.audiences` on the `CostManagementServiceConfig`)
- Claims `org_id` and `account_number` (exact names)
- Realm role `org-admin` (required for this Beta so the operator can create sources)

*NOTE: The **Client ID** you choose (for example `cost-management-metrics-operator`) is **not** the JWT audience. The gateway expects audience **`cost-management-operator`** (default on the CR). Set that value in the Audience protocol mapper even when the client ID is different.*

*IMPORTANT: Client **Settings** > **Attributes** are **not** copied into the token. Put `org_id` and `account_number` on the **service-account user**, and add **User Attribute** mappers so those values appear on the **access token**.*

Procedure

1. Open the Keycloak Admin Console in the **kubernetes** realm.
2. Confirm the top-left dropdown shows **kubernetes**.
3. Navigate to **Clients** > **Create client**.
4. Configure the following general settings:
 - Client type: **OpenID Connect**
 - Client ID: a stable name such as: `cost-management-metrics-operator`
5. Configure the following **Capability config** settings (the create-client defaults are not sufficient):
 - Client authentication: **On** (required for a **Credentials** tab and `client_secret`)
 - Authorization: **Off**
 - Standard flow: **Off**
 - Direct access grants: **Off**
 - Implicit flow: **Off**
 - Service accounts roles: **On** (`client_credentials` grant)
6. Leave login settings empty (no redirect URI) and click **Save**.
7. Open the new client and navigate to **Credentials**.
8. Copy **Client secret**. If the field is empty, click **Regenerate**. Store this value for the reporting cluster Secret in a later section.
9. Navigate to **Client scopes** and open the dedicated scope named `{client-id}-dedicated`.

10. Navigate to **Mappers**.

11. Add three mappers and, for each one, set **Add to access token** to **On**. Leave **Add to ID token** and **Add to userinfo** set to **Off**. Follow the mapper examples below to configure:

- **Mapper 1: Audience**

(Add mapper > **By configuration** > **Audience**)

- Name: `aud-cost-management-operator`
- **Included Client Audience:** `cost-management-operator`
- **Included Custom Audience:** `cost-management-operator` (set both; an Audience mapper with empty audience fields does not set `aud`)

- **Mapper 2: User Attribute `org_id`**

(Add mapper > **By configuration** > **User Attribute**)

- Name, User Attribute, and Token Claim Name: `org_id`
- Claim JSON Type: String

- **Mapper 3: User Attribute `account_number`**

(Add mapper > **By configuration** > **User Attribute**)

- Same as mapper 2, with `account_number` in every name field
- Name, User Attribute, and Token Claim Name: `account_number`
- Claim JSON Type: String

12. Open the client **Service accounts roles** tab (only when **Service accounts roles** is On).

13. Click the link to the **service-account user** (username `service-account-<client-id>`). The Users list often hides service accounts unless you filter to *Service accounts*.

14. On the service account **user** (not on the client), configure the following:

- Configure **Attributes** (`org_id` and `account_number`) with the **same** values as your `org-admin` user (for example `1234567` / `1234567`), and click **Save**.
- Navigate to **Role mapping** > **Assign role** > **Filter by realm roles** and select **org-admin**.

Verification

Request a token with the client ID and secret, then decode the access-token payload. Confirm all of the following:

- `aud` includes `cost-management-operator` (string or list)
- `org_id` and `account_number` are the values you set (bare IDs, no `org` prefix)
- `realm_access.roles` includes `org-admin`
- `iss` matches your Keycloak issuer plus `/realms/kubernetes` (or your configured realm)

Example

(Replace issuer, client ID, and secret):

```
ISSUER="https://keycloak.example.com"
TOKEN="$(curl -skS -X POST "$ISSUER/realms/kubernetes/protocol/openid-
connect/token" \
  -d grant_type=client_credentials \
  -d client_id=cost-management-metrics-operator \
  -d client_secret='<client-secret>' | jq -r .access_token)"

python3 - <<'PY' "$TOKEN"
import json, sys, base64
p = sys.argv[1].split(".")[1]
p += "=" * ((4 - len(p) % 4) % 4)
print(json.dumps(json.loads(base64.urlsafe_b64decode(p)), indent=2))
PY
```

If `aud` is missing or is only your client ID, edit the **Audience mapper** and set both **Included Client Audience** and **Included Custom Audience** to `cost-management-operator`, then request a **new** token to verify.

NOTE: After you configure the Cost Management Metrics Operator later on, a successful metrics upload through the gateway (HTTP **202 Accepted**) confirms the end-to-end path. Decoding the token is required before that step.

Install Red Hat Lightspeed cost management self-managed

Add the Cost Management Service Operator to the software catalog

Prerequisites

- You have received a self-managed cost management catalog image tag from Red Hat.
- You are logged in to your target OpenShift cluster.
- You have cluster-admin permissions.

Procedure

1. From your CLI, create a YAML file named `catalog-source.yaml`. The YAML file content should match the example below:

NOTE: For this Beta release, the catalog image is prepared for OpenShift Container Platform version 4.22.

```
apiVersion: operators.coreos.com/v1alpha1
kind: CatalogSource
metadata:
  name: koku-service-operator-catalog
  namespace: openshift-marketplace
spec:
  sourceType: grpc
  image: quay.io/project-koku/koku-service-operator-catalog:v0.0.1
  displayName: Cost Management

  publisher: Red Hat
  grpcPodConfig:
    securityContextConfig: restricted
```

2. Copy the self-managed cost management catalog image tag you received from Red Hat, and paste it into the YAML file next to the `image:` specification.
3. Save the `catalog-source.yaml` file.
4. Deploy the catalog by running `oc apply -f catalog-source.yaml` in your CLI.

Verification

Run the following commands to verify your catalog deployment:

1.

```
oc get catalogsource koku-service-operator-catalog -n openshift-marketplace \
-o jsonpath='{.status.connectionState.lastObservedState}{"\n"}'
```

Result: READY

2.

```
oc get pods -n openshift-marketplace \
-l olm.catalogSource=koku-service-operator-catalog
```

Result: 1/1 Running

3.

```
oc get packagemanifest koku-service-operator -n openshift-marketplace
```

Result: Catalog = Cost Management

If any of the above fail, run the following to investigate errors:

```
oc describe pod -n openshift-marketplace \
-l olm.catalogSource=koku-service-operator-catalog
```

Next steps

Once the catalog is deployed, you can install the Cost Management Service Operator from Software Catalog.

Install the Cost Management Service Operator

Install the Cost Management Service Operator from the OpenShift Container Platform web console.

Prerequisites

- You have already added the Cost Management Service Operator to the software catalog
- You have an OpenShift Container Platform cluster.
- You are logged in to the OpenShift Container Platform web console with an account that has cluster administrator privileges.
- Minimal hardware requirements include:
 - OpenShift 4.22 amd64/x86-64
 - CPU cores: 10
 - GiB RAM: 24
 - GiB storage: 550

Procedure

1. Log in to the **OpenShift Container Platform web console** and navigate to the **Software Catalog** page.
NOTE: If you are using OpenShift Container Platform 4.18 or earlier, navigate to the Operator Hub page.
2. Enter **Cost Management** in the search box and then select **Cost Management Service Operator**. The Cost Management Service Operator install page opens.
*NOTE: Make sure you install the Cost Management **Service** Operator (server side of cost management), not the Cost Management **Metrics** Operator (client side of cost management).*

3. Click **Install**. The Install Operator page opens.
4. Click **Install** at the bottom of the Install Operator page.
5. Click **Installed Operators** in the left-hand navigation. A page with a list of installed operators opens. Verify that the Cost Management Service Operator is installed.

Verification

- Check operator status shows **succeeded**.
- Navigate to **workloads > deployments** and confirm koku-service-operator-controller-manager shows: **1/1 ready**.

Configure the Cost Management Service Operator

Prerequisites

- You have successfully installed the Cost Management Service Operator.
- You have set up and configured the necessary external infrastructure including: PostgreSQL 16, Redis-compatible cache, Kafka, S3-compatible object storage, and OIDC.
- You have created the various secrets/credentials and fetched the corresponding routes for each piece of external infrastructure in order to add them to the configuration.

Procedure

1. Navigate to **Ecosystem > Installed Operators**.
2. Click **Cost Management Service Operator**.
3. Click **Create CostManagementServiceConfig**. The default YAML configuration file opens.
4. Fill out the necessary configuration information in the YAML file for your external infrastructure. Use the example below as a reference.

*NOTE: In OpenShift, **url** (JWKS fetch, often in-cluster http) and **issuerURL** (token iss, often public https Route) are often different values. Make sure you do **not** set both items to the same URL path.*

*NOTE: The default YAML configuration file shows you where to place your external infrastructure settings, The Cost Management Service Operator will **not** provision those external services for you.*

NOTE: The CRD default is "kubernetes" (lowercase). The realm value must match the realm name in your OIDC/Keycloak setup exactly.

Example YAML configuration file with external infrastructure information filled out:

```

kind: CostManagementServiceConfig
apiVersion: service.costmanagement.openshift.io/v1alpha1
metadata:
  name: cost-onprem
  namespace: cost-onprem
spec:
  # =====
  # REQUIRED EXTERNAL INFRASTRUCTURE CONFIGURATION
  # =====

  # PostgreSQL Database Configuration
  database:
    deploy: false
    host: 'postgresql.example.com'          # REQUIRED: External DB hostname or IP
    port: 5432
    secretName: 'cost-db-credentials'      # REQUIRED: Secret with 'username' and
    'password'
    sslMode: require                      # Options: disable, allow, prefer,
    require

  # Valkey / Redis Cache Configuration
  cache:
    deploy: false
    host: 'valkey.example.com'            # REQUIRED: External Cache hostname or
    IP
    port: 6379
    auth:
      enabled: true
      secretName: 'cost-cache-credentials' # REQUIRED: Secret with 'password'

  # Apache Kafka Configuration
  kafka:
    # Example (External): 'kafka-1.example.com:9092,kafka-2.example.com:9092'
    # Example (Internal AMQ/Strimzi): 'cost-kafka-
    bootstrap.kafka.svc.cluster.local:9092'
    bootstrapServers: 'kafka-1.example.com:9092' # REQUIRED: Comma-separated broker
    endpoints

  # Object Storage (S3-compatible) Configuration
  objectStorage:
    endpoint: 's3.example.com'            # REQUIRED: S3 endpoint host (without
    protocol prefix)
    port: 443
    useSSL: true
    secretName: 'customer-object-storage-credentials' # REQUIRED: Secret with
    AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY
    buckets:
      koku: 'koku-bucket'                # REQUIRED: Pre-created S3 bucket name

  # Keycloak / OIDC Authentication Configuration
  auth:

```

```

keycloak:
  # Example (External/Route): 'https://keycloak-keycloak.<APPS_DOMAIN>'
  # Example (Internal Service): 'https://keycloak-
service.keycloak.svc.cluster.local:8443'
  url: 'https://keycloak-keycloak.<APPS_DOMAIN>' # REQUIRED: Keycloak base
HTTPS URL
  envoy:
    image:
      repository: registry.redhat.io/openshift-service-mesh/proxyv2-rhel9
      tag: '2.6'
    replicas: 2

# =====
# DEFAULT OPERATOR SERVICES & IMAGE CONFIGURATION
# =====

global:
  clusterDomain: apps.cluster.local
  storageClass: ''
costManagement:
  api:
    image:
      repository: quay.io/project-koku/koku
      tag: 422f758
    replicas: 1
  celery:
    workers:
      costModel:
        replicas: 1
      default:
        replicas: 1
      ocp:
        replicas: 2
      priority:
        replicas: 1
      summary:
        replicas: 2
  listener:
    replicas: 2
  masu:
    image:
      repository: quay.io/project-koku/koku
      tag: 422f758
    replicas: 1
  ingress:
    image:
      repository: quay.io/iop/ingress
      tag: sha-6dde23d
  kruize:
    image:
      repository: quay.io/redhat-services-prod/kruize-autotune-tenant/autotune
      tag: d1f0140

```

```
replicas: 1
monitoring:
  enabled: true
rbac:
  api:
    replicas: 1
  image:
    repository: quay.io/project-koku/insights-rbac
    tag: 34e25ed
  worker:
    replicas: 1
ros:
  enabled: false
  image:
    repository: quay.io/redhat-services-prod/insights-management-tenant/insights-ocp-resource-optimization/ros-ocp-backend
    tag: ae8c990
  processor:
    replicas: 1
ui:
  app:
    image:
      repository: quay.io/insights-onprem/koku-ui-onprem
      tag: d64d393ede6ab625e53c63202a6bcd6fa48e718
  oauthProxy:
    image:
      repository: registry.redhat.io/rhceph/oauth2-proxy-rhel9
      tag: v7.6.0
  replicaCount: 1
```

5. Click **Create**.

Configure the Cost Management UI OAuth Client

Configure the OAuth client that supports browser sign-in to the Cost Management UI.

This is a one-time identity-provider task. Create users and assign their organization attributes separately after the client is configured.

The Cost Management UI includes oauth2-proxy, which redirects a browser to your identity provider for sign-in. After a successful login, the provider redirects the browser back to oauth2-proxy. oauth2-proxy exchanges the authorization code using this client's ID and secret, then maintains the user session.

The provider must issue an access token that contains the user's `org_id` and `account_number` claims. The Cost Management gateway rejects a token that is missing either claim.

Prerequisites

- Collect the following values from the CostManagementServiceConfig and your OpenShift cluster:
 - The configured Keycloak realm. The default is `kubernetes`.
 - The CostManagementServiceConfig name, namespace, and cluster apps domain.
 - The UI callback URL: `https://{cr-name}-ui-{namespace}.{apps-domain}/oauth2/callback`
 - The UI base URL: `https://{cr-name}-ui-{namespace}.{apps-domain}`
 - The values configured in `spec.auth.keycloak.audiences`. Defaults are `cost-management-operator` and `cost-management-ui`.

NOTE: The operator must be able to reach `spec.auth.keycloak.url` for JSON Web Key Set discovery. If the token `iss` value uses a different public URL, set `spec.auth.keycloak.issuerURL` to that issuer URL.

Procedure

1. In the Keycloak Admin Console, select the realm configured for Cost Management and create a client with these settings.

Setting	Value
Client type	OpenID Connect
Client ID	cost-management-ui, or another stable client ID that you also store in the Kubernetes Secret
Client authentication	Enabled; this is a confidential client
Standard flow	Enabled
Direct access grants	Disabled in production
Valid redirect URI	The exact UI callback URL shown above
Web origins	The UI base URL

2. Copy the client secret from the client's Credentials tab. Do not put this secret in the CostManagementServiceConfig itself.

3. Configure access token claims:

- Organization claims: Create two User Attribute protocol mappers on the UI client, or put them on a client scope that is assigned to the UI client.
 - For each user, set both attributes. Use the same stable values for every user in the same Cost Management organization. Each value must use only letters, numbers, periods, underscores, or hyphens, and must be no more than 128 characters long.

Mapper setting	org_id	account_number
Mapper type	User Attribute	User Attribute
User Attribute	org_id	account_number
Token Claim Name	org_id	account_number
Claim JSON Type	String	String
Add to access token	Enabled	Enabled
Add to ID token	Disabled	Disabled
Multivalued	Disabled	Disabled

- Audience claim: Ensure the access token `aud` claim includes at least one value from `spec.auth.keycloak.audiences`. A dedicated client scope with Audience mappers is recommended when several Cost Management clients share the same setup.
 - For the default CMSC settings, add these audiences to the access token:
 - `cost-management-operator`
 - `Cost-management-ui`
- Administrator role: Create the `org-admin` realm role if it does not already exist. Assign this realm role only to people who should administer Cost Management or delegate User Access roles. Ensure the UI client includes realm roles in access tokens, so `org-admin` appears in `realm_access.roles`.
 - An `org-admin` role in a client, or a Keycloak group with the same name, is not equivalent to the required realm role.

4. Store the client credentials in Kubernetes:

- Create a Secret in the same namespace as the CostManagementServiceConfig. Its default name is `{cr-name}-ui-oauth-client`; use `spec.ui.oauthClientSecretRef.name` if you choose a different name.

```
apiVersion: v1
kind: Secret
metadata:
  name: {cr-name}-ui-oauth-client
  namespace: {namespace}
type: Opaque
stringData:
  client-id: cost-management-ui
  client-secret: replace-with-the-client-secret
```

- Apply the Secret, then allow the operator to reconcile. The UI Deployment's oauth2-proxy container reads these two values; neither key may be empty.

5. Validate the configuration:

- Open the Cost Management UI URL in a private browser window.
- Confirm that you are redirected to the identity provider and then returned to the UI after sign-in.
- Inspect the resulting access token with your approved token-inspection method. Confirm that it includes `org_id`, `account_number`, a matching `aud` value, and `org-admin` in `realm_access.roles` for an administrator.
- Confirm that an administrator can open Cost Management and User Access.

NOTE: If `UIReady=False` has reason `OAuthClientSecretMissing`, check the Secret name and its non-empty `client-id` and `client-secret` keys. If login returns to the UI with an error, verify that the registered redirect URL exactly matches the UI callback URL.

Create the UI OAuth client Secret for the Cost Management Service Operator

*NOTE: The Service Operator can be installed and the CostManagementServiceConfig can be created **before** this Secret exists. Until the Secret is present with `client-id` and `client-secret`, `UIReady` stays `False` with reason `OAuthClientSecretMissing`. The UI `oauth2-proxy` path is not fully ready until then. That is expected, **not** a failed install.*

After you configure the UI OAuth client in Keycloak, the Cost Management Service Operator needs a Kubernetes Secret in the **same namespace** as your `CostManagementServiceConfig`. The operator reads this Secret to configure the UI `oauth2-proxy`.

Do **not** put the client secret in the CR spec. This Secret is **not** the same as the Cost Management Metrics Operator service-account Secret. The Metrics Operator Secret uses keys `client_id` and `client_secret` (underscores). The UI Secret uses `client-id` and `client-secret` (hyphens). This Secret is **not** the same as creating the UI client in Keycloak.

Workflow options

- **Option A (recommended):** Create the project namespace > apply the `CostManagementServiceConfig` > create this Secret > confirm `UIReady=True`.
- **Option B:** Create the project namespace > create this Secret first (you must already know `metadata.name` and `metadata.namespace` for the CR) > apply the `CostManagementServiceConfig`.

IMPORTANT: The Secret must always live in the same namespace as the CR.

Prerequisites

- You completed the UI OAuth client procedure in Keycloak in the **kubernetes** realm (or the realm you set in `spec.auth.keycloak.realm` on the CR).
- You applied your `CostManagementServiceConfig` (you need `metadata.name` and `metadata.namespace`).
- You copied the UI client secret from Keycloak: **Clients** > your UI client (for example `cost-management-ui`) > **Credentials**. **Secret name** Default: `{CR_NAME}-ui-oauth-client`, where `{CR_NAME}` is `metadata.name` on the CR.

Example: if `metadata.name` is `cost-onprem`, the Secret name is `cost-onprem-ui-oauth-client`. Use a different name only if you set `spec.ui.oauthClientSecretRef.name` on the CR.

Required keys

Key	Value
<code>client-id</code>	Client ID from Keycloak (often <code>cost-management-ui</code>)
<code>client-secret</code>	Secret from Keycloak Credentials

Procedure (OpenShift CLI)

Replace `<NAMESPACE>`, `<CR_NAME>`, and the client secret value.

```
oc -n <NAMESPACE> create secret generic <CR_NAME>-ui-oauth-client \
  --from-literal=client-id=cost-management-ui \
  --from-literal=client-secret='<from Keycloak → Clients → cost-management-ui →
  Credentials>'
```

To create or update without failing when the Secret already exists:

```
oc -n <NAMESPACE> create secret generic <CR_NAME>-ui-oauth-client \
  --from-literal=client-id=cost-management-ui \
  --from-literal=client-secret='<client-secret>' \
  --dry-run=client -o yaml | oc apply -f -
```

Procedure

1. In the OpenShift web console, switch to project `<NAMESPACE>` (the same project as your `CostManagementServiceConfig`).
2. Navigate to **Workloads > Secrets**.
3. Click **Create > Key/value secret**.
4. In **Secret name**, enter `<CR_NAME>-ui-oauth-client`.
5. Add key `client-id` with the UI client ID from Keycloak.
6. Click **Add key/value**, and add key `client-secret` with the value from Keycloak **Credentials**.
7. Click **Create**.

Verification

1. On your `CostManagementServiceConfig`, the **UIReady** condition becomes **True**. While the Secret is missing or incomplete, **UIReady** stays **False** with reason `OAuthClientSecretMissing`.
2. Optional CLI check:

```
oc get costmanagementserviceconfig <CR_NAME> -n <NAMESPACE> \
  -o jsonpath='{.status.conditions[?(@.type=="UIReady")].status} reason={.status.conditions[?(@.type=="UIReady")].reason}'
```

NOTE: You need **UIReady=True** before you can log in through the cost management UI in **Access the self-managed instance of cost management**.

Verify the Cost Management Service Operator

If you require additional verification to confirm that the Cost Management Service Operator is configured correctly, navigate to Installed **Operators > Cost Management Service Operator** ****and scroll down to Conditions****. Compare with the statuses listed below::

You're configuration is successful with the following statuses:

Field / condition	Expected (beta, Cost-only)	Notes
status.phase	Ready	Human summary; prefer conditions for detail
Available	True	Primary "stack is up" (AllComponentsReady)
UIReady	True	Required for UI login
Progressing	False	False = reconcile finished (not an error)
Degraded	False	False = not degraded (not an error)
ROSEnabled	False	Expected when ros.enabled: false

Dependency & component conditions (all should be True)

Condition	Expected	What it means (one line)
DiscoveryComplete	True	Cluster domain / storage class discovered
DatabaseReady	True	DB Secret + reachability
CacheReady	True	Cache Secret + reachability
KafkaReady	True	Bootstrap + required topics
StorageReady	True	S3 credentials + buckets
AuthenticationReady	True	OIDC JWKS reachable
SchemaUpToDate	True	Migrations complete

RBACReady	True	RBAC API up
RBACWorkerReady	True	RBAC worker up
IngressReady	True	Upload ingress up
GatewayReady	True	API gateway up

Next steps

- Once the Cost Management Service Operator is installed and configured, install the Cost Management Metrics Operator (if you haven't already) and configure it.

Client clusters setup

Install the Cost Management Metrics Operator

Install the Cost Management **Metrics Operator** from the OpenShift Container Platform web console so you can import OpenShift usage data into cost management.

Prerequisites

- You have an OpenShift Container Platform cluster.
- You are logged in to the OpenShift Container Platform web console with an account that has cluster administrator privileges.
- You can access your self-management cost management UI.

Procedure

1. Log in to the OpenShift Container Platform web console and navigate the **Software Catalog** page.
2. Enter **Cost Management** in the search box and then select **Cost Management Metrics Operator**. The Cost Management Metrics Operator install page opens.
3. Click **Install**. The Install Operator page opens.
4. Click **Install** at the bottom of the Install Operator page.

Verification

- From the OpenShift Container Platform web console, navigate to **Ecosystem > Installed Operators**.
- Verify that the **Cost Management Metrics Operator** appears in the list with status **Succeeded**.

Next steps

- After installing the Cost Management Metrics Operator, configure it to upload data to your Cost Management Service Operator with service account authentication.

Create service accounts to authenticate the clients

Prerequisites

- You have created a service account for your Cost Management Metrics Operator in Keycloak, following *Create keycloak Service Account for Cost Management Metrics Operator* above.
- You have installed the Cost Management Metrics Operator.

Procedure

1. In the OpenShift Container Platform web console, click the tab **Workloads** and then click **Secrets**.
2. In the **Secrets** window, click the Create drop-down and then select **Key/value secret**. You will make two keys: one for your **Client ID** and one for your **Client Secret**.
3. Enter the following information in the **Create key/value secret** window:
 - In the **Secret Name** box, enter `service-account-auth-secret`.
 - In the **Key** box, enter `client_id`.
In the **Value** box for the first key `client_id`, upload the value for your authorized SA created in keycloak or paste it into the text box. This is the client ID that you saved when you made your service account.
4. Click **Add Key/Value** to add the second key/value pair for your client secret.
5. In the **Key** box, enter `client_secret`:
6. In the **Value** box for the second key `client_secret`, upload the **Value** for your authorized SA created in keycloak or paste it into the text box.
7. After you verify that the key/value details for the secret are correct, click Create to complete the creation of your service account authorization secret.
8. Copy the name of your secret. You will use it in the following section.

Configure clients to upload data to the server

Prerequisites

- You have created a secret for your service account credentials

Procedure

1. From the OpenShift Container Platform web console, click the **Operators** tab and then click **Installed Operators**.
2. Find the **Cost Management Metrics Operator** and click its name.
3. Click the tab **Cost Management Metrics Config** and then click the configuration file in **Name**. The default name is `costmanagementmetricscfg-sample`.
4. Click the tab **YAML** to open the file.
5. Locate the spec section in the YAML file:
6. Change the **api_url**: to match the gateway-host URL for your self managed cost management
7. Under **authentication** change:
 - **token_url**: `https://<keycloak-host>/realms/<realm>/protocol/openid-connect/token` to match your keycloak endpoint as follows
 - **type**: **service-account**.
 - Insert a new line for **secret_name**. Enter the secret that you created.
8. Under **source** section
 - Change **source_path** to `/api/cost-management/v1/`
9. Under **upload** section:
 - Change **validate_cert**: `false`

Example:

```
spec:
  api_url: 'https://<gateway-host>' #HTTPS origin of this cost management gateway
  authentication:
    token_url: 'https://<keycloak-host>/realms/<realm>/protocol/openid-
connect/token'
    type: service-account
    secret_name: operator-service-account
  packaging:
    max_reports_to_store: 30
    max_size_MB: 100
  prometheus_config:
    collect_previous_data: true
    context_timeout: 120
    disable_metrics_collection_cost_management: false
    disable_metrics_collection_resource_optimization: false
    service_address: 'https://thanos-querier.openshift-monitoring.svc:9091'
    skip_tls_verification: false
  source:
    check_cycle: 1440
    create_source: false
    name: my-cluster-name
    sources_path: /api/cost-management/v1/
  upload:
    ingress_path: /api/ingress/v1/upload
    upload_cycle: 360
    upload_toggle: true
    validate_cert: false
```

10. Click **Save**.

Verification

- Navigate to **Operators > Installed Operators > Cost Management Metrics Operator**.
- Click the **CostManagementMetricsConfig** tab and select your configuration.
- Click **YAML** and verify that authentication shows type: service-account and your secret name.

Next steps

- Use the self-managed instance of the cost management service.

Access the self-managed instance of Red Hat Lightspeed cost management

After installing and configuring the cost management operators with service account authentication, you are ready to access the self-managed instance of cost management.

*NOTE: You can only log in to self-managed cost management if your user already exists in Keycloak with **org_id** and **account_number** set. If you need administrator permissions, ensure your user has been assigned the org-admin **realm** role before attempting to log in.

Prerequisites

- You have an OpenShift Container Platform cluster.
- You are logged in to the OpenShift Container Platform web console with an account that has cluster administrator privileges.
- You have installed and configured the Cost Management Service Operator and the Cost Management Metrics Operator.
- You have created a service account for your Cost Management Metrics Operator through Keycloak as part of your own external infrastructure.
- You have created a Secret for your service account credentials.

Procedure

1. From the OpenShift Container Platform web console, navigate to **Networking > Routes**.
2. From the **Project** dropdown menu, select the project namespace for your self managed cost management deployment
3. Locate and click on the corresponding link under **Location** to login to Cost management.
4. Log in to the cost management UI using your new org-admin user. The self-managed instance of the cost management UI appears.

Next steps

- Refer to the rest of the cost management documentation to guide you through how to analyze, understand, and optimize your resource costs across hybrid cloud infrastructure. The following topics may be useful:
 - Using cost models
 - Managing cost data using tagging
 - Analyzing your cost data
 - Visualizing your costs using cost explorer

Product(s) Red Hat Hybrid Cloud Console Red Hat Lightspeed
Red Hat OpenShift Container Platform

Category Install **Article Type** General

Comments

Login to see comments



Quick Links

Help

Site Info

Related Sites

About

Red Hat Subscription Value

About Red Hat

Red Hat Jobs



[About Red Hat](#)

[Jobs](#)

[Events](#)

[Locations](#)

[Contact Red Hat](#)

[Red Hat Blog](#)

[Inclusion at Red Hat](#)

[Cool Stuff Store](#)

[Red Hat Summit](#)

Copyright © 2026 Red Hat

[Privacy statement](#)

[Terms of use](#)

[All policies and guidelines](#)

[Digital accessibility](#)

[Cookie preferences](#)